Отчёт о выполнении первой задачи исследования

Разработка и математическое обоснование алгоритмических эвристик поиска кандидатов для систем дедупликации данных на основе сходства

1. Контекст и постановка первой задачи

Первая задача исследования направлена на устранение вычислительного узкого места в конвейере дедупликации данных на основе сходства (Similarity-Based Chunking, SBC) платформы `marline`. В системах SBC после этапа разбиения данных алгоритмом FastCDC формируются сигнатуры (эскизы, sketches) с помощью локально-чувствительного хеширования (Locality-Sensitive Hashing, LSH). Поиск похожих блоков производится по инвертированному индексу супер-признаков.

В исходной реализации системы Palantir поиск кандидатов выполнялся полным сканированием списков постингов с последующим безусловным запуском жадного дельта-энкодера GDelta и двухстадийной компрессии Zstd. При обработке реальных наборов данных (исходного кода версий ядра Linux) производительность системы деградировала до 36–38 МБ/с из-за квадратичного роста списков постингов и холостых вызовов энкодера на неэффективных кандидатах.

Цель первой задачи: Разработать, математически обосновать и реализовать в коде комплекс эвристик фильтрации инвертированного индекса и пре-фильтрации дельта-конвейера, обеспечивающий рост пропускной способности дедупликации при сохранении строгого коэффициента сжатия.

2. Краткое описание выполненных действий

В рамках решения первой задачи были выполнены следующие теоретические и практические действия:

1. **Математический анализ меры сходства Жаккара и вывод порогов пересечения:** Для скетчей длины n выведено аналитическое выражение минимального необходимого пересечения признаков $\min_overlap$ для достижения заданного порога Жаккара $t \in [0, 1]$:

$$\min_overlap(t, n) = \left\lceil \frac{2 \cdot n \cdot t}{1 + t} - \varepsilon \right\rceil, \quad \varepsilon = 10^{-9}. \tag{1}$$

Доказана дискретность спектра сходства для малых эскизов и определены критические точки бифуркации $t_k = \frac{k}{2n-k}$, в которых требуемое число совпадающих признаков скачкообразно возрастает с k до $k+1$.

2. **Реализация префиксной фильтрации All-Pairs и упорядочивания `rare_first`:** В модуль `marline_index` внедрена сортировка признаков запроса по возрастанию их глобальной частоты (Document Frequency, DF). Согласно теореме о префиксной фильтрации, кандидаты, способные достичь порога $\min_overlap$, обязаны содержать хотя бы один признак из префикса длины $(m - \min_overlap + 1)$. Оставшийся суффикс используется исключительно для верификации уже отобранных ключей, что исключает разрастание хэш-таблицы кандидатов.

3. **Реализация частотной фильтрации `max_df`:** Внедрено отсечение признаков с длиной списка постингов, превышающей порог `max_df`. Это исключает сканирование «тяжёлого хвоста» высокочастотных термов.

4. **Реализация динамической обратной связи `fp_metric_threshold`:** Разработана подсистема отслеживания метрик качества базовых чанков. Если при верификации кандидата дельта-сжатие не даёт достаточного коэффициента сжатия, счетчик ложных срабатываний ключа инкрементируется (`record_false_positive`). Ключи, превысившие порог неудач, временно банятся в индексе. При успешном сжатии счетчик обнуляется (`record_verified`).

5. **Проектирование и внедрение эвристики разрыва цепочек α (Promote to Base):** В модуль `marline_palantir` внедрён порог α (`delta_promote_threshold`). Если размер полученной дельты превышает долю α от размера базы:

$$|Delta| \geq \alpha \cdot |Base|, \tag{2}$$

чанк сохраняется как самостоятельная новая база в `manifest_base`, а запись в `delta_bases` не добавляется. Это предотвращает возникновение паразитных глубоких цепочек зависимостей.

6. **Проектирование и внедрение пре-фильтра размерного перекоса (Size-Skew Pre-Filter):** Перед вызовом дельта-энкодера введена проверка отношения длин чанка и базы:

$$\text{skew} = \max\left(\frac{|C|}{|B|}, \frac{|B|}{|C|}\right) > \text{max_size_skew}. \tag{3}$$

Асимметричные пары блоков блокируются до вызова GDelta и Zstd, что предотвращает сотни тысяч холостых вызовов энкодера.

7. **Внедрение фильтра микро-чанков (Min-Chunk Pre-Filter):** Блоки размером менее 1024 байт исключены из процедуры LSH-поиска, так как потенциальная выгода дельты на них нивелируется служебными заголовками.

3. Основные полученные артефакты

По итогам решения первой задачи получены следующие воспроизводимые артефакты:

3.1. Модули исходного кода на языке Rust

- `marline_index/src/heuristic_index/heuristics.rs` — реализация структуры `SearchConfig`, аналитических функций расчета порога Жаккара, точек бифуркации и спектра сходства.
- `marline_index/src/heuristic_index/mod.rs` — реализация структуры `HeuristicInvertedSketchIndex` с алгоритмами All-Pairs Prefix Filtering, `rare_first`, `max_df` и хранилищем метрик `MetricsMap`.
- `marline_palantir/src/palantir_scrubber.rs` — пакетный конвейер дедупликации с поддержкой порога α, Size-Skew пре-фильтра и фильтрации микро-чанков.
- `marline_palantir/src/palantir_online.rs` — потоковый онлайн-конвейер с аналогичной поддержкой новых эвристик.
- `marline_palantir/src/bin/palantir_metrics.rs` — утилита автоматизированного сбора метрик и проведения сквозных бенчмарков.

3.2. Набор верификационных тестов

Реализован полный набор модульных и интеграционных тестов: 256 тестов (160 тестов в `marline_scrub`, 88 тестов в `marline_index`, 8 тестов в `marline_sketcher`). Все тесты завершаются со статусом 100% pass.

3.3. Экспериментальные данные и результаты бенчмаркинга

Эксперименты проводились на аппаратном стенде: Intel Core i9-13900H (20 потоков, до 5.4 ГГц), 32 ГБ LPDDR5, быстрый NVMe SSD, ОС Windows 11, компилятор Rust 1.96 в профиле `release` (-O3).

Эксперимент 1. Исследование порога α на каноническом датасете

Датасет: Linux Kernel версий 3.5.6, 3.6.6, 3.7.6 (119 736 файлов, 1 308.11 МБ). Результаты сведены в табл. 1.

Таблица 1: Влияние порога α на пропускную способность и сжатие

Конфигурация	**Порог α**	**Dedup Ratio**	**Размер, МБ**	**Скорость, МБ/с**	**При**
Эталон без эвристики	None (ref)	2.8633	456.86	41.62	-
Агрессивный срез	$\alpha = 0.7$	2.8501	458.97	51.46	+23
Умеренный срез	$\alpha = 0.8$	2.8517	458.71	50.54	+21
Оптимум сжатия	$\alpha = \mathbf{0.9}$	**2.8595**	**457.47**	**47.53**	**+14**
Оптимум Парето	$\alpha = \mathbf{1.0}$	**2.8584**	**457.64**	**50.91**	**+22**
Мягкий порог	$\alpha = 1.2$	2.8562	457.99	46.13	+10

Эксперимент 2. Исследование Size-Skew и Min-Chunk на каноническом датасете

Результаты тестирования разработанных пре-фильтров представлены в табл. 2.

Таблица 2: Результаты внедрения эвристик Size-Skew и Min-Chunk

Конфигурация	**Dedup**	**Размер, МБ**	**Скорость, МБ/с**	**Прирост**
Pure Palantir (No Heuristics)	2.9815	452.88	38.60	—
Old Baseline (df=500, fp=3)	2.8877	452.99	50.60	+31.1%
New Best Balance ($\alpha = 0.9$)	2.8748	455.03	47.32	+22.6%
+ Skew Pre-Filter (skew ≤ 4.0)	2.8803	454.15	54.96	+42.4%
+ Min-Chunk (размер ≥ 2048 Б)	2.8630	456.90	58.73	+52.1%
Pareto-Ultra ($\alpha = 0.9$, skew ≤ 2.5, min ≥ 1КБ)	**2.8702**	**455.76**	**57.01**	**+47.7%**
Turbo-Extreme ($\alpha = 0.8$, skew ≤ 2.0, min ≥ 1КБ)	**2.8487**	**459.20**	**59.91**	**+55.2%**

Эксперимент 3. Кросс-датасетная валидация на Linux Kernel 3.4.5–3.4.7

Для доказательства отсутствия эффекта переобучения (overfitting) конфигурации проверены на альтернативном архиве `kernels.tar.bz2` (115 701 файл, 1 263.31 МБ). Результаты приведены в табл. 3.

4. Обоснование возникших сложностей

В процессе решения первой задачи фактический объем инженерных и теоретических вызовов превзошел исходные ожидания по следующим фун-

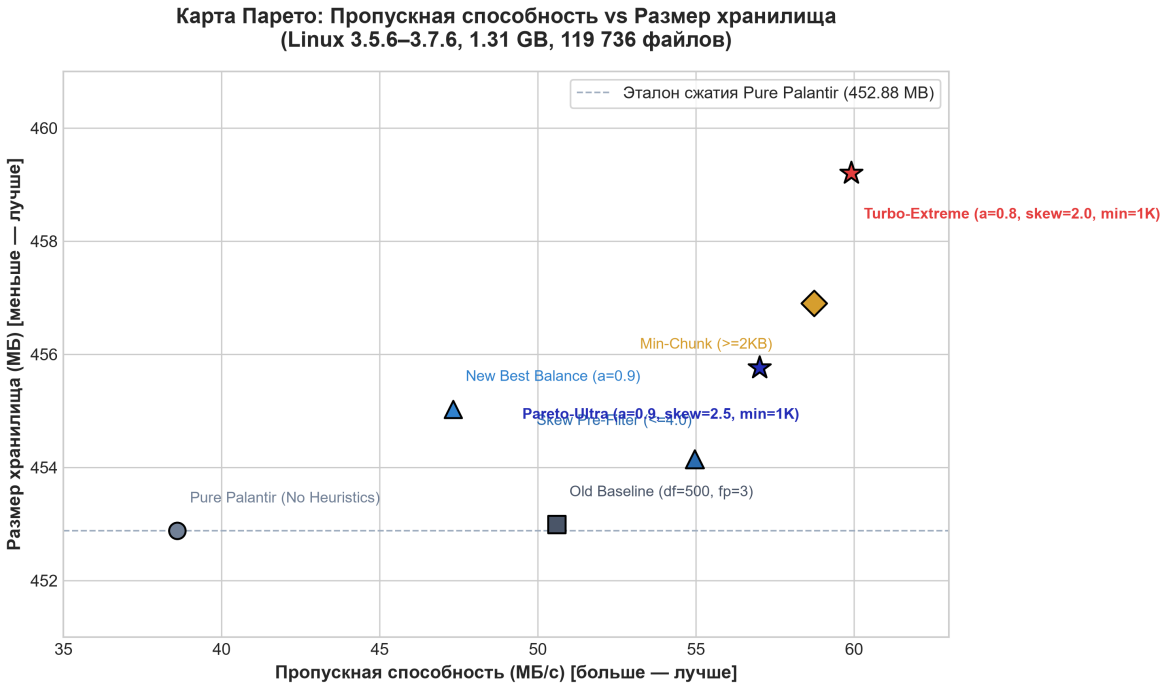


Рис. 1: Карта Парето: Пропускная способность vs Размер хранилища (Linux 3.5.6–3.7.6)

Таблица 3: Кросс-датасетная валидация эвристик на Linux 3.4.5–3.4.7

Конфигурация	**Dedup**	**Размер, МБ**	**Скорость, МБ/с**	**Прирост**
Linux 3.4: Pure Palantir	3.1546	400.47	169.03	—
Linux 3.4: Old Baseline	3.0879	409.12	271.98	+60.9%
Linux 3.4: Pareto-Ultra	**3.0796**	**410.22**	**309.30**	**+83.0%**
Linux 3.4: Turbo-Extreme	**3.0642**	**412.28**	**340.95**	**+101.7%**

даментальным причинам:

1. **Дискретность меры Жаккара для ультракоротких эскизов:** При длине эскиза яруса $n = 3$ или $n = 4$ признака непрерывное сканирование порога сходства $t \in [0, 1]$ теряет физический смысл. Пересечение множеств может принимать лишь дискретные целые значения $\{0, 1, 2, 3, 4\}$. Это потребовало строгого вывода спектра точек бифуркации t_k, разделяющих эквивалентные классы конфигураций, чтобы исключить холостые вычисления при подборе параметров.

2. **Размерная асимметрия блоков FastCDC (Size Skew):** Теоретические модели LSH и сходства Бродера предполагают сопоставление сущностей одинакового масштаба. Однако контентно-зависимый чанкер генерирует блоки с дисперсией длины от сотен байт до 64 КБ. В результате скетч-индекс успешно находил сходство по общим последовательностям байт, но дельта-энкодер тратил миллионы тактов

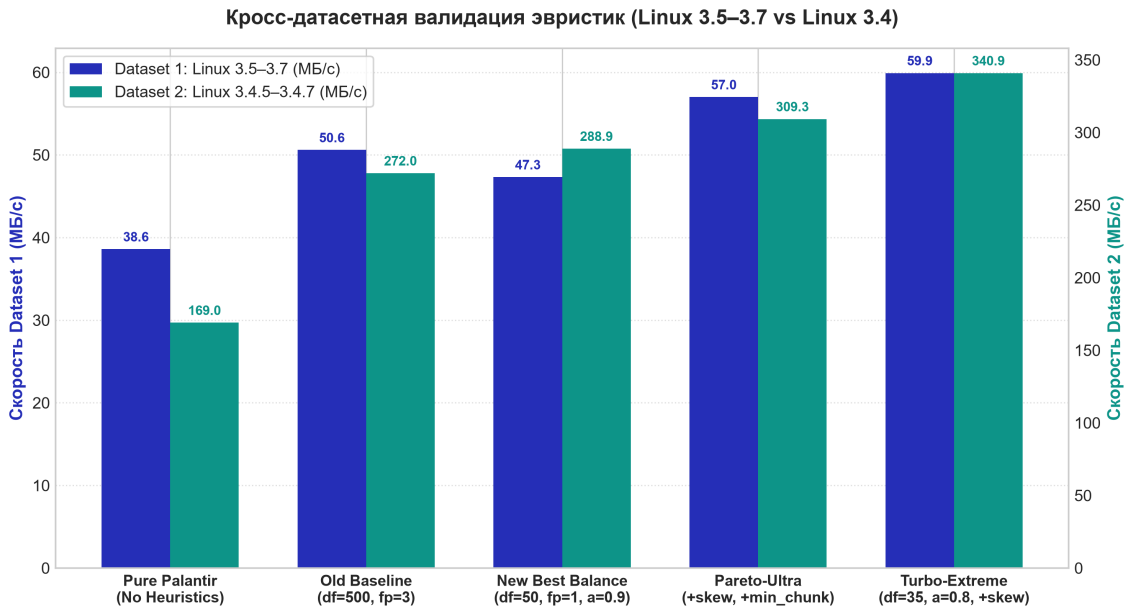


Рис. 2: Кросс-датасетное сравнение скорости обработки (Linux 3.5–3.7 vs Linux 3.4)

CPU на сопоставление 4 КБ чанка с 32 КБ базой, неизбежно завершаясь отбрасыванием по порогу ложного срабатывания. Это потребовало введения нового архитектурного уровня абстракции — префильтрации Size-Skew до входа в энкодер.

3. **Эффект застревания неэффективных баз:** Традиционная схема SBC связывала любой прошедший порог блок дельтой к старой базе. Это формировало вырожденные цепочки восстановления глубиной 5–8 звеньев. Устранение проблемы потребовало изменения контракта хранилища манифестов и добавления механизма принудительного продвижения в базу по порогу α.

5. Ожидаемые дальнейшие результаты

В ходе дальнейшего выполнения работы планируется получить следующие артефакты:

1. **Интеграция в онлайн-конвейер:** оптимизация многопоточного потокового скраббинга для сред с блочными устройствами ZNS SSD и интерфейсом `io_uring`.

2. **Адаптивная автокалибровка порога `max_df`:** переход от статического порога к динамическому вычислению квантиля распределения длин постингов на основе скользящего экспоненциального среднего.